

Lesson 8: Cross-Site Scripting (XSS)

Marcin Kurzyna

Lecture Goals

This lecture introduces:

- What Cross-Site Scripting (XSS) is,
- How attackers inject malicious code,
- Different types of XSS attacks,
- Why browsers execute dangerous scripts,
- Basic defenses against XSS.

1 What Is XSS?

Cross-Site Scripting (XSS) is a web security vulnerability. It happens when a website displays user input without proper protection so attackers can inject JavaScript code into webpages.

2 Why XSS Is Dangerous

With XSS attacks, attackers may:

- Steal cookies,
- Steal user sessions,
- Redirect users,
- Modify webpages,
- Display fake login forms,
- Spread malware.

XSS attacks happen inside the victim's browser.

3 Basic Example

Imagine a webpage that displays user comments.

```
name = input("Your name: ")
print("Hello " + name)
```

If the user enters:

```
<script>alert("Hacked!")</script>
```

the browser may execute the script.

4 What Happens?

The webpage becomes:

```
Hello <script>alert("Hacked!")</script>
```

The browser interprets:

- HTML,
- JavaScript,

instead of plain text.

5 Reflected XSS

Reflected XSS happens when:

- Malicious code comes from the current request,
- The server immediately returns it in the response.

Example:

- Search forms,
- URL parameters.

6 Stored XSS

Stored XSS happens when:

- Malicious code is saved in a database,
- Other users later load the infected page.

Example:

- Forums,
- Comments,
- Social media posts.

Stored XSS is often more dangerous.

7 Unsafe HTML Output

Dangerous example:

```
comment = input("Comment: ")
print("<div>" + comment + "</div>")
```

Problem:

- User input becomes HTML code.

8 Escaping Output

Safer idea:

- Treat user input as text,
- Not as HTML.

Example:

```
import html

comment = input("Comment: ")
safe = html.escape(comment)

print("<div>" + safe + "</div>")
```

Now dangerous characters are escaped.

9 Input Validation

Applications should validate:

- Length,
- Format,
- Allowed characters.

Example:

- Reject JavaScript tags,
- Reject suspicious input.

10 Why Browsers Execute Scripts

Browsers are designed to:

- Execute JavaScript,
- Render HTML dynamically.

Without proper protections:

- Browsers cannot distinguish safe and malicious code.

11 Real-World Examples

XSS vulnerabilities have affected:

- Social media websites,
- Online stores,
- Forums,
- Banking applications.

Even large companies make mistakes.

Summary

In this lecture, we covered:

- What XSS is,
- How browsers execute malicious scripts,
- Reflected, Stored XSS,
- Escaping, input validation and basic browser protections.

12 Exercises

1. Explain why this code is dangerous:

```
name = input("Name: ")
print("<h1>" + name + "</h1>")
```

2. What will happen if the user enters:

```
<script>alert("Attack")</script>
```

3. Improve this code using escaping:

```
message = input("Message: ")
print("<p>" + message + "</p>")
```

4. What is the difference between:

- Reflected XSS,
- Stored XSS?

5. Challenge: Design a safe comment system that:

- Displays user comments,
- Prevents script execution,
- Protects other users from attacks.